

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение высшего образования

Санкт-Петербургский национальный исследовательский университет ИТМО

Мегафакультет трансляционных информационных технологий

Факультет информационных технологий и программирования

Отчет по проектной работе “Tetris AI”

По дисциплине “Машинное обучение”

Выполнил студент группы М3303

Лукин Даниил Валерьевич

Проверила

Власова Елизавета Кирилловна



УНИВЕРСИТЕТ ИТМО

Санкт-Петербург

2026

Проработка теории

Правила игр

1. Сетка Тетриса состоит из 10 ячеек в ширину и 22 ячеек в высоту, при этом верхние 2 ряда скрыты.
2. Все тетрамино (фигурки тетриса) начнут движение из середины двух верхних рядов.
3. Всего существует 7 тетрамино: "I", "O", "J", "L", "S", "Z", "T".
4. Для рандомизации следующих элементов используется генератор случайных чисел
5. Допускается одна фигура, на которую игрок может посмотреть вперед (игрок знает, какой будет следующая фигура).

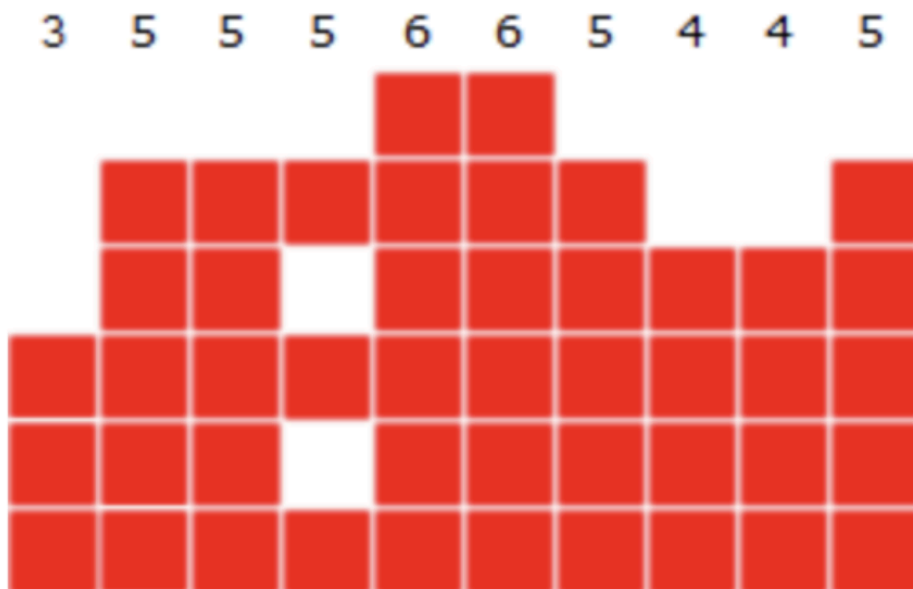
Что конкретно мы хотим

Наша цель - расчистить как можно больше линий и, следовательно, совершить как можно больше ходов. Если проще говоря не допустить того чтобы у нас элементы достигли верхней границы. Для достижения этой цели наш ИИ будет определять наилучший ход для заданной фигуры Тетриса, перебирая все возможные варианты. Он вычисляет оценку для каждого возможного хода и выбирает ход с наилучшей оценкой в качестве следующего хода.

Что по параметрам

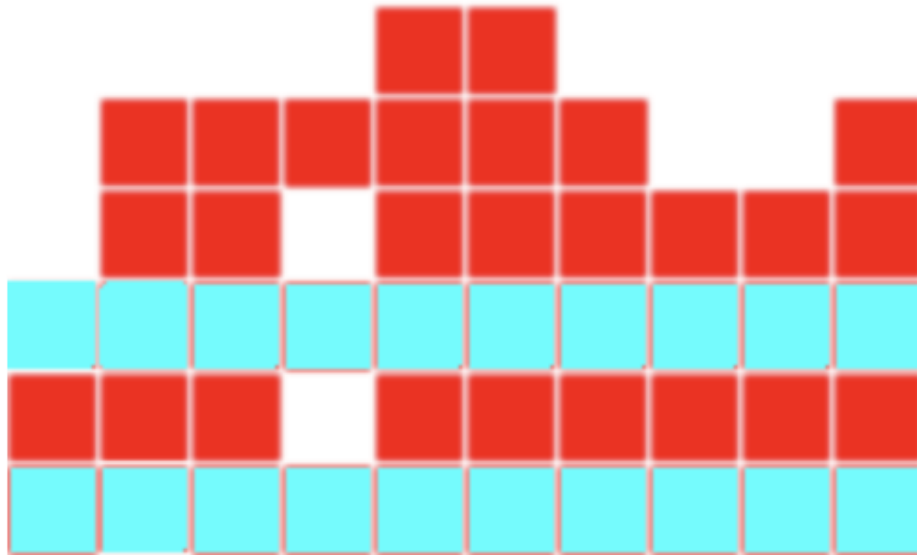
Суммарная высота

Этот эвристический метод позволяет определить высоту ячейки сетки. Для вычисления общей высоты мы суммируем высоты каждого столбца (расстояние от самой высокой ячейки в каждом столбце до нижней границы ячейки сетки). В примере ниже общая высота будет аж 48:) Нам хочется минимизировать это значение, потому что меньшая суммарная высота означает, что мы можем разместить больше элементов в сетке, прежде чем достигнем ее вершины.



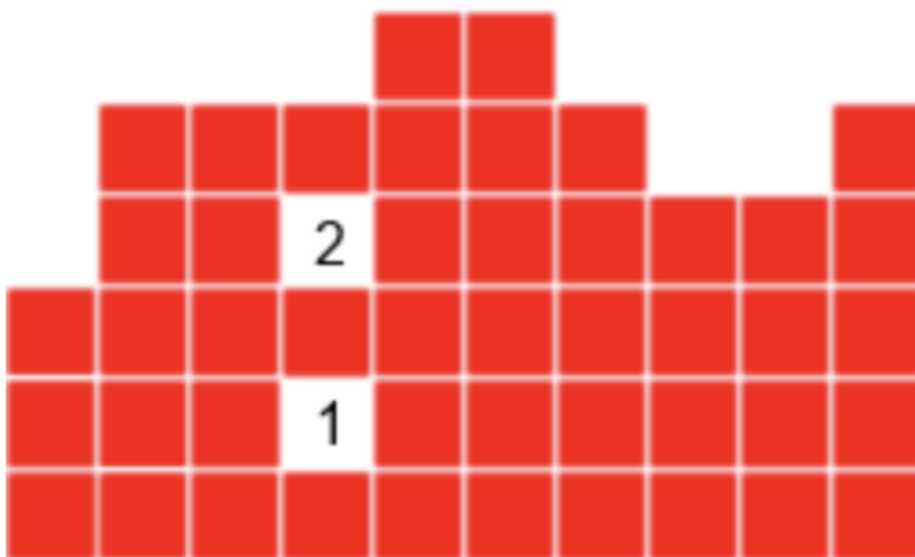
Полные строки

Это, пожалуй, наиболее интуитивно понятная эвристика из всех четырех. Она просто показывает количество полных линий в сетке. Нам нужно будет максимизировать количество полных линий, потому что цель помним наша цель всё-таки не допустить дохождения до вершины



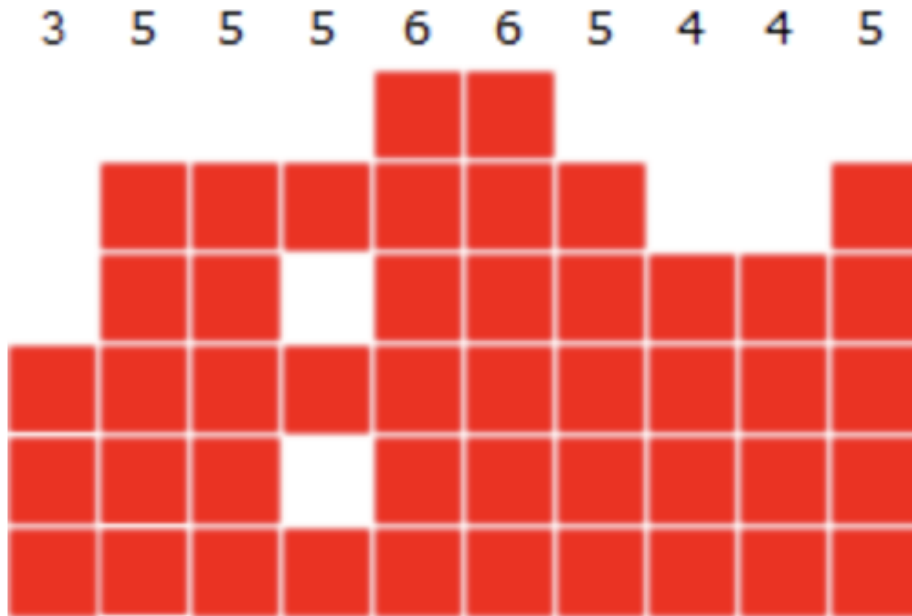
Пустоты

Пустота определяется как пустое пространство (да я знаю русский язык :), над которым находится по крайней мере одна плитка в том же столбце. Такую штука довольно сложно устранить, потому что нам придется очистить все линии над ним, прежде чем мы сможем добраться до него и заполнить. Поэтому нам нужно свести количество таких пробелов к минимуму.



Гладкость/Неровность

Мы не хотим допускать колодец. Почему? Потому что мы будем зависеть от “Г”, что становится проблемой так как при большом колодце нас не спасёт только одна палка. Неровность сетки показывает вариативность высоты ее столбцов. Она вычисляется путем суммирования абсолютных разностей между двумя соседними столбцами (проще говоря модуль соседних). У варианта ниже будет неровность в 6



Надо начинать думать (ну или понимать матан)

Алгоритм

Вот примерно так вычисляем оценку сетки, используя линейную комбинацию наших четырех эвристических алгоритмов.

$$a \times \text{AggregateHeight} + b \times \text{CompleteLines} + c \times \text{Hole} + d \times \text{Bumpiness}$$

Как легко догадаться a, c, d будут отрицательные. Почему? Мы же хотим их минимизировать отсюда логично что вес должен быть отрицательным. Ну и наоборот для b - будет положительная.

Да здравствует матан (ну или моя боль)

На первых порах легко понять, что такой набор будет просто вектором (A, B, C, D) . Как я отметил ранее у нас они функция оценки будет линейная (если забыли каждая отдельная штука есть линейная функция :(). Чтобы понять это с математической точки зрения, сначала перепишем функцию оценки в контексте векторов следующим образом:

$$f(|(x)_1) = |(p) \times |(x)$$

Предположим, мы хотим сравнить два хода, ход 1 и ход 2, чтобы определить, какой ход “лучше”. Также, что ходы 1 и 2 дают значения $|(x)_1$ и $|(x)_2$ соответственно. Чтобы проверить, лучше ли ход 1, чем ход 2, нам нужно проверить, выполняется ли условие. Если это условие не выполняется, то легко заключить, что ход 2 лучше или так же хорош, как ход 1. (Ну и поехали)

$$f(|(x)_1) - f(|(x)_2) = |(p) \times |(x)_1 - |(p) \times |(x)_2 = |(p) \times (|(x)_1 - |(x)_2)$$

Здесь мы видим, что результат сравнения остается неизменным при масштабировании параметров - мы можем умножить $|(p)$ на любую положительную действительную (скалярную) константу, и знак вышеуказанной разницы останется тем же (хотя величина изменится - но это не имеет значения), в этом случае решение не изменится. Все векторы параметров, направленные в одном направлении, дают эквивалентные результаты. Таким образом, достаточно рассматривать только один вектор для каждого направления. Это оправдывает ограничение поиска только точками на поверхности единичной 3-сферы, поскольку поверхность сферы охватывает все возможные направления.

Функция “поджигания”

Для каждого вектора параметров мы запускаем ИИ на 100 играх, в каждой из которых не более 500 фигур. Мы принимаем значение приспособленности $\text{fitness}(|(p))$ любого вектора параметров $|(p)$ равным общему количеству очищенных линий. Наихудшее возможное значение приспособленности равно 0, в этом случае ИИ не очищает ни одной линии ни в одной игре. (ну тупой игрок короче).

Родительские особи отбираются для размножения с использованием турнирного отбора. Мы случайным образом отбираем 10% популяции, и две наиболее приспособленные особи из этой подгруппы участвуют в кроссинге для получения нового потомства. Этот процесс повторяется до тех пор, пока количество полученных новых потомков не достигнет 30% от размера по сути 300 сущностей.

Взвешенное среднее значение пересечения

$$|(p') = |(p)_1 \times \text{fitness}(|(p)_1) + |(p)_2 \times \text{fitness}(|(p)_2)$$

Реализация

Архитектура нейронной сети

Для оценки позиции на игровом поле используется простая нейронная сеть прямого распространения (feedforward). Сеть принимает на вход 4 эвристических признака и выдает одно число - оценку качества позиции.

Слой	Размер	Параметры
Входной	4 нейрона	-
Скрытый	3 нейрона	12 весов + 3 смещения
Выходной	1 нейрон	3 веса + 1 смещение

Table 1: Архитектура нейронной сети

Итого: 19 параметров (16 весов + 3 смещения)

В качестве функции активации используется ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \max(0, x)$$

Реализация эвристических функций

Вычисление высоты столбцов

Функция `get_column_heights` является базовой для нескольких других эвристик. Она вычисляет высоту каждого столбца игрового поля - расстояние от верхней заполненной ячейки до дна.

```
std::vector<int> brain_t::get_column_heights(const std::vector<std::vector<int>>& grid)
noexcept {
    std::vector<int> column_heights;
    for (int col = 0; col < 10; col++) {
        int height = 20; // максимальная высота игрового поля
        for (int row = 4; row < 24; row++) { // пропускаем 4 скрытых ряда сверху
            if (grid[row][col] != 0 && grid[row][col] != 1) {
                break; // нашли первую заполненную ячейку
            } else {
                height--;
            }
        }
        column_heights.push_back(height);
    }
    return column_heights;
}
```

Суммарная высота

```
int brain_t::get_aggregate_height(const std::vector<std::vector<int>>& grid) noexcept {
    int sum = 0;
    std::vector<int> heights = get_column_heights(grid);
    for (int i = 0; i < 10; i++) {
        sum += heights[i];
    }
    return sum;
}
```

Полные строки

```
int brain_t::get_completed_lines(const std::vector<std::vector<int>>& grid) noexcept {
    int completed_lines = 0;
    for (int l = 0; l < 24; l++) {
        if (find(grid[l].begin(), grid[l].end(), 0) == grid[l].end()) {
            completed_lines++;
        }
    }
    return completed_lines;
}
```

Пустоты (Holes)

```
int brain_t::get_holes(const std::vector<std::vector<int>>& grid) noexcept {
    int holes = 0;

    for (int row = 23; row > 3; row--) { // идём снизу вверх
        const std::vector<int> zeros = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
        if (grid[row] == zeros) {
            return holes; // достигли пустой строки - выходим
        }

        for (int col = 9; col >= 0; col--) {
            bool covered = false;
            if (grid[row][col] == 0) { // нашли пустую ячейку
                // проверяем, есть ли над ней заполненные ячейки
                for (int r = row - 1; r > 3; r--) {
                    if (grid[r][col] != 0) {
                        covered = true;
                        break;
                    }
                }
                if (covered) {
                    holes++;
                }
            }
        }
    }
    return holes;
}
```

Неровность (Bumpiness)

```
int brain_t::get_bumpiness(const std::vector<std::vector<int>>& grid) noexcept {
    std::vector<int> heights = get_column_heights(grid);
    int bumpiness = 0;
    for (int i = 0; i < 9; i++) { // 9 пар соседних столбцов
        bumpiness += std::abs(heights[i] - heights[i + 1]);
    }
    return bumpiness;
}
```

Вычисляем модуль разности высот между соседними столбцами. Суммируем все 9 разностей (для 10 столбцов). Минимальное значение - 0 (ровная поверхность), максимум - значительно выше.

Выбор лучшего хода

Функция `best_move` реализует двухходовой lookahead (заглядывание вперёд): для текущей и следующей фигуры перебираются все возможные позиции.

```
std::string brain_t::best_move(const tetris::grid_t& grid) noexcept {
    std::string bestMove;
    std::string bestRotation;
    float max_score = static_cast<float>(-pow(10.0, 5));
    const int n_rotations = tetris::SHAPE_ROTATIONS[grid.get_piece().get_n()];
    tetris::grid_t starting_grid_t = grid;
    std::vector<std::string> possible_moves = {"llll", "lll", "ll", "l", "rrrrr", "rrrr",
"rrr", "rr", "r", ""};

    for (int r = 0; r < n_rotations; r++) {
        for (int m = 0; m < 10; m++) {
            tetris::grid_t grid = starting_grid_t;
            // Применяем вращение и смещение
            // ...
            grid.gravity(3);
            const std::vector<int> heuristics = {
                get_aggregate_height(grid.get_matrix()),
                get_completed_lines(grid.get_matrix()),
                get_holes(grid.get_matrix()),
                get_bumpiness(grid.get_matrix())
            };
            float score = forward(params, heuristics);
            // Lookahead для следующей фигуры
            // ...
        }
    }
    return bestMove + bestRotation;
}
```

Почему используем lookahead? Зная следующую фигуру, мы можем принять решение, которое создаст более благоприятную позицию для следующего хода.

Возврат значения: Функция возвращает строку, где символы 'l'/'r' означают смещение влево/вправо, а последняя цифра - количество вращений.

Оператор кроссовера

Кроссовер (crossover) объединяет параметры двух родителей с учётом их приспособленности:

```
`brain_t brain_t::crossover(brain_t partner_param) noexcept {
    brain_t offspring;
    neural_net_t p1 = this->params;
    neural_net_t p2 = partner_param.params;
    float f1 = this->score;
    float f2 = partner_param.score;

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            offspring.params.layer1[i][j] = (f1 * p1.layer1[i][j] + f2 * p2.layer1[i][j])
                                             / (f1 + f2 + pow(10, -10));
        }
        offspring.params.biases1[i] = (f1 * p1.biases1[i] + f2 * p2.biases1[i])
                                       / (f1 + f2 + pow(10, -10));
    }
    return offspring;
}
```

Почему именно взвешенное среднее? Это позволяет потомку наследовать больше генов от более успешного родителя. Добавление `pow(10, -10)` предотвращает деление на ноль.

Генетический алгоритм

Многопоточная оценка

```
void genetic_t::fit() {
    const unsigned num_threads = std::thread::hardware_concurrency();

    while (true) {
        std::vector<std::future<std::pair<size_t, brain_t>>> futures;

        for (size_t b = 0; b < population_.agents_count(); b++) {
            futures.push_back(std::async(std::launch::async, [this, b]() {
                brain_t result = run_games(population_.get_agents()[b]);
                return std::make_pair(b, result);
            }));
        }

        if (futures.size() >= num_threads) {
            for (auto& f : futures) {
                auto [agent_idx, result] = f.get();
                population_.get_agents()[agent_idx] = result;
            }
            futures.clear();
        }
        population_.generate_new();
    }
}
```

Почему batch обработка? Ограничиваем количество одновременно работающих потоков числом ядер CPU.

Результаты

Обучение

Обучение происходит в реальном времени. Лучшие параметры сохраняются в файл. Процесс продолжается до ручной остановки.

Вывод

Могу сказать, что реализация на сpp позволила мне лучше понять ML (именно взаимодействие нейронов).